CRS-SSD: Fine-Grained Composite Risk Scoring for Speculative Safety-Aware Decoding

Atharv Nair, Raghusrinivasan Venkatesan,
a3nair@ucsd.edu, r1venkatesan@ucsd.edu

Abstract

Large language models remain vulnerable to jailbreak attacks even after alignment. We build on *Speculative Safety-Aware Decoding* (SSD), which repurposes the draft model in speculative decoding as a safety-conscious filter, switching between token-set intersection and union modes based on a match ratio. Our primary contribution is a *Composite Risk Score* (CRS) that replaces this coarse binary switch with a per-token continuous risk signal combining KL divergence, refusal-token mass, distribution mismatch, and entropy gap, enabling finer-grained and more responsive mode control. We additionally explore two secondary extensions: CRS-modulated hidden-state steering and integration with *Alignment-Augmented Speculative Decoding* (AASD) Wang et al. [2025a]. On a custom high-ASR evaluation set of 70 harmful and 30 benign prompts, CRS with ablation-optimized weights achieves **20.0%** attack success rate, a 14.3 percentage point reduction over vanilla decoding with zero over-refusal on benign inputs. Steering does not improve over CRS alone, and AASD requires a decaying blend schedule to avoid generation collapse. Component ablations identify KL divergence and refusal-token mass as the primary risk signals. This work focuses on safety; efficiency evaluation is left for future work. Code is available at `https://github.com/codeFafnir/safety-Aware-speculative-decoding-strategies`.

1 Introduction

Aligned large language models (LLMs) are routinely deployed for assistive tasks, yet they remain susceptible to *jailbreak* attacks that elicit harmful or unsafe behavior through adversarial prompts, roleplay framing, or nested narrative techniques Cheng et al. [2023], HuggingFace and community [2024]. Defenses that rely on retraining or heavy fine-tuning are costly and may not generalize to new attack types. *Inference-time* interventions that operate purely during decoding offer a lightweight alternative, requiring no changes to the target model's weights.

Speculative decoding uses a small draft model to propose tokens that a larger target model verifies in parallel, reducing latency Leviathan et al. [2023], Chen et al. [2023]. Wang et al. [2025b] introduced **Speculative Safety-Aware Decoding (SSD)**, which repurposes this setup for safety: a safety-conscious draft model and the target model produce token distributions at each step, and a *match ratio* over a fixed window (e.g., 7 tokens) determines whether to sample from the *intersection* or *union* of their top-c token sets. A high match ratio is interpreted as benign. A low match ratio triggers a more permissive *Union* mode that allows the draft's refusal tokens into the sample space. SSD improves safety while retaining inference speedup, but its binary, window-based switching is coarse and can miss early-token jailbreak signals or over-refuse on long benign responses.

Our primary contribution is the **Composite Risk Score (CRS)**: we replace the binary match-ratio switch with a continuous per-token signal combining (i) match mismatch, (ii) KL divergence between draft and target distributions, (iii) entropy gap, and (iv) refusal-token mass. Mode switching is driven

by CRS every token rather than every 7-token window, enabling finer-grained and more responsive safety control. We additionally explore three secondary extensions:

- **Hidden-state steering (exploratory):** We inject the target model's last hidden state into the draft's MLP layers, scaling injection strength by CRS. The projection is trained with *refusal distillation* on harmful prompts. Results show this does not improve over CRS alone on our dataset.
- **AASD integration (exploratory):** We integrate SSD with AASD-style Wang et al. [2025a] alignment sampling. A naive constant blend schedule causes generation collapse; a decaying schedule recovers performance but does not improve over CRS without AASD.
- **Training-free extensions:** Contrastive system prompts and a perplexity gate (forcing Union mode from token 1 on high-perplexity prompts) that add no parameters and require no training. The PPL gate effect on our subset is inconclusive and discussed in Section 8.

We evaluate on a custom dataset of 70 harmful prompts (40 DeepInception, 30 JailbreakBench-wrapped) and 30 benign XSTest prompts, with Qwen3Guard as the safety classifier. Our main findings are as follows. CRS with ablation-optimized weights achieves **20.0% ASR**, a 14.3 pp reduction over vanilla, with zero over-refusal. Steering without CRS slightly *worsens* safety relative to the SSD baseline (30.0% vs. 28.6%), since fixed-scale injection adds noise without risk-gating. AASD logit blending with a frozen prefill prior causes generation collapse (57.1% ASR), but an exponentially decaying blend schedule fully recovers at 24.3% ASR with zero over-refusal. Template-wrapped jailbreaks remain a notable failure mode.

The rest of the paper is organized as follows. Section 2 summarizes preliminary experiments and the research pivot. Section 3 reviews related work. Section 4 formalizes the problem and SSD baseline. Section 5 describes our extensions. Section 6 and Section 7 present setup and main results. Section 8 presents ablation studies. Section 9 shows qualitative examples. Section 10 discusses findings and Section 11 concludes.

2 Preliminary Work and Research Pivot

Our midterm report explored a different research direction: whether post-training quantization (PTQ) degrades safety alignment in instruction-tuned LLMs. This was motivated by recent work arguing that 4-bit compression can perturb alignment properties Wee et al. [2025], Chen et al. [2025]. We evaluated Qwen3-4B-Instruct in full-precision (FP) versus a 4-bit GGUF variant across three conditions: SafetyBench classification accuracy, AdvBench refusal rates under normal and shallow jailbreak conditions, and a BeaverTails-based safety-neuron analysis.

Table 1 summarizes the quantization results. FP and GGUF are effectively tied on SafetyBench (79% vs. 79% accuracy). On AdvBench, both models achieve $>98\%$ refusal under no jailbreak; under a shallow prompt-space jailbreak, differences remain small (flip rate 1.15%, GGUF slightly more compliant). The safety gap on BeaverTails (harmful refusal minus benign refusal) differed by only -0.02 between FP and GGUF. These results confirm that *quantization alone is not a meaningful driver of safety regression* in our setting.

Table 1: Quantization (FP vs. 4-bit GGUF) safety comparison on Qwen3-4B-Instruct. Results show negligible differences across all three evaluation conditions, motivating a pivot to decoding-time jailbreak defenses.

Condition	Metric	FP	GGUF	Δ
SafetyBench (11k prompts)	Accuracy↑	0.786	0.792	$+0.006$
AdvBench baseline (520)	Refusal rate↑	0.989	0.992	$+0.003$
AdvBench + jailbreak	Refusal rate↑	0.994	0.987	-0.007

In the safety-neuron analysis, we identified 61,747 ranked MLP neuron candidates (concentrated in layers 34–35 of Qwen3-4B) using activation contrast between harmful and benign BeaverTails prompts. A causal patching demo on the top-200 neurons left refusal rates unchanged (0.9425 before and after), indicating that naive mechanistic interventions are insufficient for controlling refusal behavior at this scale.

Pivot rationale. Three findings from the midterm directly motivated this work: (1) quantization effects are modest relative to jailbreak-driven vulnerabilities, so future-proofing safety requires addressing decoding robustness; (2) static activation patching does not change refusal, and a dynamic per-token risk signal that modulates sampling is more promising; (3) the literature gap identified at midterm (no method combining per-token risk estimation with adaptive mode switching) directly motivated our CRS design. We therefore shifted to extending SSD Wang et al. [2025b] with a Composite Risk Score, hidden-state steering, and AASD integration, as described in Section 5.

3 Related Work

Jailbreak defenses. Defenses range from input filtering Robey et al. [2023], paraphrase or repetition checks Zhang et al. [2024], to inference-time decoding strategies. SafeDecoding Xu et al. [2024] adjusts the sampling distribution using a safety expert model (e.g., CensorTune) by blending base and expert logits and sampling from the intersection of top-k tokens; it does not use speculative decoding. Our baseline SSD Wang et al. [2025b] instead uses a single draft model for both speed and safety and switches between intersection/union based on match ratio.

Speculative decoding. Leviathan et al. [2023] and Chen et al. [2023] introduced speculative decoding with a draft model proposing multiple tokens and the target verifying in parallel. Extensions include adaptive draft length Agrawal et al. [2024], Lu et al. [2025] and risk-bounded acceptance Bai et al. [2024]. AASD Wang et al. [2025a] blends draft logits with the target's prefill prior using KV-cache-based incremental decoding; we integrate this alignment sampling approach with SSD.

Representation steering. Activation addition Turner et al. [2023] steers models at inference by adding contrastive activation differences. Steering pretrained drafters during speculative decoding Berdoz et al. [2025] injects target information into the draft; we adopt a similar idea with a CRS-modulated refusal-distillation projection.

4 Problem Setup and SSD Baseline

4.1 Setting

We have a *target* LLM P_{t} (e.g., Qwen2.5-7B-Instruct) and a smaller *draft* LLM P_{d} (e.g., Qwen2.5-1.5B-Instruct). Both are given the same user prompt (possibly with different system prompts). At each decoding step t, they produce token distributions $p_{\mathrm{t}}(\cdot \mid x_{<t})$ and $p_{\mathrm{d}}(\cdot \mid x_{<t})$. The goal is to produce a sequence that (i) stays close to the target's behavior on benign prompts (utility) and (ii) refuses or avoids harmful content on jailbreak prompts (safety), without retraining the target.

4.2 SSD Algorithm (Baseline)

SSD Wang et al. [2025b] uses the draft as a safety-conscious partner. At each step:

1. Compute top-c token sets $\mathcal{T}_{\mathrm{t}}$, $\mathcal{T}_{\mathrm{d}}$ from the two distributions.
2. Maintain a *match ratio* β over the last b tokens: the fraction of steps where the *sampled* token was in the draft's top-c.
3. If $\beta > \beta_{\mathrm{th}}$: *Intersection* mode. Sample from $\mathcal{T}_{\mathrm{t}} \cap \mathcal{T}_{\mathrm{d}}$ with composite score $p_{\mathrm{t}} + \alpha_I(p_{\mathrm{d}} - p_{\mathrm{t}})$.
4. If $\beta \leq \beta_{\mathrm{th}}$: *Union* mode. Sample from $\mathcal{T}_{\mathrm{t}} \cup \mathcal{T}_{\mathrm{d}}$ with $p_{\mathrm{t}} + \alpha_U(p_{\mathrm{d}} - p_{\mathrm{t}})$, allowing the draft's refusal tokens into the support.

Annealing (Algorithm 2 in the original paper) decays β_{th} and α_I when the scheme stays unchanged to limit over-refusal on long benign replies.

4.3 Limitations of Binary Match-Ratio Switching

The match ratio is computed every b tokens and is binary in effect (above or below threshold). Thus (i) the first b tokens have no safety signal from agreement, (ii) slow-building jailbreaks (e.g., narrative) may keep β high until harm is already underway, and (iii) the signal is an average over the window rather than a per-token risk. Our CRS addresses these by providing a continuous, per-token risk that can drive both mode choice and steering strength.

5 Method

5.1 Composite Risk Score (CRS)

At each token step t, we define a scalar **Composite Risk Score** $r_t \in [0, 1]$ from four normalized signals:

- **Match mismatch** $1 - \mathbb{I}[\hat{x}_t \in \mathcal{T}_\mathrm{d}]$: disagreement between the (previously) sampled token and the draft's top-c (we use a running indicator).
- **KL divergence** $\mathrm{KL}(p_\mathrm{d} \| p_\mathrm{t})$: large values indicate the draft distribution diverges from the target (e.g., draft refuses, target does not).
- **Entropy gap** $H(p_\mathrm{t}) - H(p_\mathrm{d})$: draft more confident than target can indicate an unusual (e.g., harmful) context.
- **Refusal mass** $\sum_{v \in \mathcal{R}} p_\mathrm{d}(v)$: probability the draft assigns to a fixed set of refusal tokens (e.g., "I cannot", "I'm sorry").

We normalize each component to $[0, 1]$ (e.g., min-max over a short history or clip) and take a convex combination:

$$r_t = w_1(1 - \mathrm{match}) + w_2\,\overline{\mathrm{KL}} + w_3\,\overline{\Delta H} + w_4\,\overline{\mathrm{refusal}}, \tag{1}$$

The score is smoothed over a window of 3 steps. When $\bar{r}_t > \tau_\mathrm{crs}$ (e.g., 0.2), we switch to *Union* mode, enlarging the candidate set to include the draft's refusal tokens and applying a safety-weighted composite score, mirroring SSD's logic that low draft-target agreement signals risk. When $\bar{r}_t \leq \tau_\mathrm{crs}$, we use *Intersection* mode, restricting sampling to tokens both models endorse. Thus CRS replaces the 7-token binary match-ratio switch with a per-token, continuous gate.

5.2 Hidden-State Steering

Inspired by Berdoz et al. [2025], who show that injecting target model representations into a draft during speculative decoding can guide the draft's behavior without retraining, we add a *steering* module that injects the target's last-layer hidden state into the draft's MLP layers. The key intuition is that the target model, being larger and better aligned, encodes stronger refusal signals in its hidden states; by projecting these into the draft's representation space, we can bias the draft's token distribution toward refusal precisely when the CRS indicates elevated risk.

Projection. Let $h_\mathrm{t} \in \mathbb{R}^{d_\mathrm{t}}$ be the target's last hidden state and d_d the draft's hidden size. We learn a linear map $\mathrm{proj} : \mathbb{R}^{d_\mathrm{t}} \to \mathbb{R}^{d_\mathrm{d}}$ by *refusal distillation*: on harmful prompts where the target refuses, we minimize the KL between the draft's next-token distribution (after receiving a projected vector) and the target's, or a hidden-state loss (e.g., MSE between draft hidden states and projected target hidden states). Training uses only harmful/refusal data; training on general helpfulness data was found to increase ASR.

Inference. We unit-normalize h_t before projecting, compute $g_t = \mathrm{proj}(h_\mathrm{t}/\|h_\mathrm{t}\|)$, and add g_t scaled by η_t to the draft's MLP outputs at all layers (or a fraction of layers). The scale is CRS-modulated:

$$\eta_t = \eta_0\big(1 + \lambda\,\bar{r}_t\big), \tag{2}$$

so steering is stronger when CRS is high.

5.3 Integration with AASD

We combine SSD with the AASD pipeline Wang et al. [2025a] for efficiency and alignment:

1. **Prefill & KV cache:** Run a single full forward pass for both models on the prompt; keep target and draft KV caches and the target's *prefill logits* at the last prompt position.
2. **Alignment sampling with decaying schedule:** At each step t, draft logits are blended with the (fixed) target prefill prior: $\ell_\mathrm{d} \leftarrow (1 - \lambda_t)\ell_\mathrm{d} + \lambda_t\,\ell_\mathrm{t}^{\mathrm{prefill}}$, where $\lambda_t = \lambda_0 \cdot \gamma^t$ decays exponentially (e.g., λ_0=0.3, γ=0.92). This biases the draft toward the target's initial response direction at early tokens while fading out before the prior diverges from the growing context. A constant λ causes generation collapse (Section 7).

3. **Incremental decoding:** For each new token, only the new token is fed in; KV caches are updated (O(1) per step instead of O(n) full recompute).
4. **Conditional verification:** In intersection mode, if $p_{\mathrm{t}}(\hat{x}_t) < \tau \cdot \max_v p_{\mathrm{t}}(v)$, we discard the token and resample from the union set with α_U, avoiding low-probability target tokens that could be harmful.

SSD's intersection/union and match-ratio (or CRS) logic run on top of this pipeline; the draft logits used for token sets and composite sampling are the aligned draft logits above.

5.4 Training-Free Extensions

- **Contrastive system prompts:** The draft receives a strong safety-focused system prompt; the target receives a standard "helpful assistant" prompt. This widens the draft-target distribution gap on harmful prompts without fine-tuning, making match ratio (and CRS) more discriminative.
- **Perplexity-gated early Union:** Before generation, we compute prompt perplexity under the draft. If it exceeds a threshold τ_{ppl} (default 150), we force Union mode from token 1 to handle GCG-style and other high-perplexity jailbreaks where the match-ratio window would otherwise be inactive for the first b tokens.

6 Experimental Setup

Models. Target: Qwen/Qwen2.5-7B-Instruct. Draft: Qwen/Qwen2.5-1.5B-Instruct. Guard: Qwen/Qwen3Guard-Gen-0.6B. All models in fp16. Max new tokens: 256. Hardware: 2× NVIDIA A6000 (48GB each).

Datasets. We use a custom evaluation set to obtain meaningful ASR spread: 70 harmful prompts (40 DeepInception-style nested fiction + 30 JailbreakBench behaviors wrapped in DAN/AIM/roleplay-style templates) and 30 benign prompts from XSTest (surface-dangerous wording, benign intent) to measure over-refusal. All safety labels are from Qwen3Guard (no keyword heuristics).

Methods. (1) **Vanilla**: target only, greedy decoding. (2) **SSD**: match-ratio switching every b=7, contrastive prompts, PPL gate. (3) **SSD-CRS**: CRS-driven mode switching, no steering, default weights (w_1=w_2=0.3, w_3=0.1, w_4=0.3). (4) **SSD-CRS+Steering**: CRS + hidden-state steering (projection trained with refusal distillation). (5) **SSD-Steering-Only**: same steering and projection as SSD-CRS+Steering but with vanilla SSD binary match-ratio switching (no CRS), serving as an ablation to isolate the contribution of CRS vs. steering. (6) **SSD-CRS-AASD**: CRS with ablation-optimized weights + AASD decaying blend (λ_0=0.3, γ=0.92).

Metrics. Attack Success Rate (ASR) on harmful prompts (lower is better) and over-refusal rate on benign XSTest (lower is better). We also report per-source ASR (DeepInception, JBB-wrapped) where available.

7 Results

Table 2 summarizes the main results on the Qwen2.5-7B-Instruct target. Vanilla ASR is 34.3%. SSD reduces overall ASR to 28.6%, a 5.7 pp improvement. Replacing the binary switch with CRS at default weights yields a further reduction to 24.3%, bringing the total improvement to 10.0 pp over vanilla. **SSD-CRS with ablation-optimized weights achieves the best result at 20.0% ASR**, a 14.3 pp reduction versus vanilla and 8.6 pp versus the SSD baseline, with zero over-refusal. On the 35-prompt ablation subset (Section 8), dropping the match-mismatch and entropy-gap terms eliminates spurious Union-mode triggers, reducing the union-step fraction from 31.3% to 3.5% while preserving the safety gain. Adding CRS-modulated hidden-state steering gives 25.7% ASR. Steering without CRS yields 30.0%, confirming that CRS is the primary driver of the safety improvement. AASD logit blending with a frozen prefill prior collapses generation, shrinking responses to a mean of 27 words and pushing ASR to 57.1%. A decaying schedule ($\lambda_t = \lambda_0 \cdot \gamma^t$, γ=0.92) with ablation-optimized CRS weights fully recovers to 24.3% ASR with zero over-refusal and 184-word responses. Over-refusal is 0% for vanilla, SSD, SSD-CRS with ablation-optimized weights, and SSD-CRS-AASD. Default-weight CRS variants show 3.3% over-refusal.

Table 2: Attack Success Rate (ASR) and over-refusal. Lower ASR is better. "—" indicates not evaluated on that split.

Method	ASR % ↓	Refusal % ↑	Over-ref % ↓	Resp len
Vanilla	34.3	64.3	0.0	179
SSD baseline	28.6	70.0	0.0	200
SSD-Steering-Only	30.0	68.6	0.0	201
SSD-CRS (default weights)	24.3	74.3	3.3	205
SSD-CRS (best weights)	**20.0**	**80.0**	**0.0**	184
SSD-CRS + Steering	25.7	72.9	3.3	205
SSD-AASD (collapsed, frozen λ)	57.1	42.9	0.0	27
SSD-CRS-AASD (decaying λ)	**24.3**	**74.3**	**0.0**	184

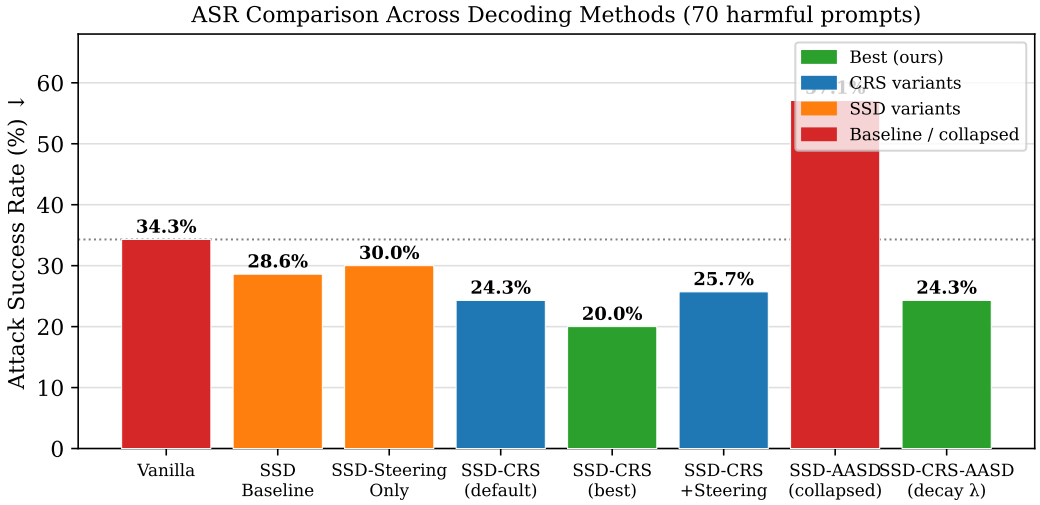


Figure 1: ASR across all evaluated methods (70 harmful prompts, Qwen2.5-7B-Instruct target). Green bars mark our best-performing variants; red marks baselines and the collapsed AASD configuration.

AASD: collapse and fix. AASD-style logit blending with a constant blend weight toward a frozen prefill prior causes generation collapse: responses shrink to ∼27 words and ASR jumps to 57.1%, worse than vanilla. An exponentially decaying schedule $\lambda_t = \lambda_0 \cdot \gamma^t$ (λ_0=0.3, γ=0.92) combined with best CRS weights fully recovers: 24.3% ASR and 184-word responses.

8 Ablation Studies

8.1 CRS vs. Steering

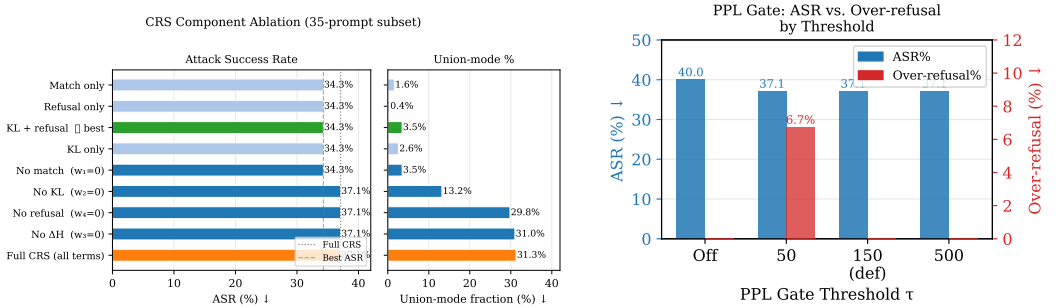
SSD-Steering-Only (steering with vanilla match-ratio, no CRS) achieves 30.0% ASR, which is slightly *worse* than the SSD baseline (28.6%) and substantially worse than SSD-CRS (24.3%). Adding steering without CRS provides no safety benefit and can slightly degrade performance, because the fixed-scale injection adds noise uniformly regardless of context risk. *CRS is the primary driver* of the safety gain. When CRS is present, adding steering does not further improve safety: SSD-CRS+Steering yields 25.7% ASR versus 24.3% for SSD-CRS alone (higher is worse). This suggests CRS-modulated injection adds noise that slightly undermines the CRS-alone configuration, though the 1.4 pp gap is within the range of small-sample variance on this dataset.

8.2 CRS Component Ablation

We ablated each CRS term on a 35-prompt harmful subset (Table 3). Key findings: (1) Removing ΔH (entropy gap, w_3) has zero effect, confirming it is dead weight in this configuration. (2) The match-

Table 3: CRS component ablation on 35-prompt harmful subset (20 DI + 15 JBB). Baselines: Vanilla=57.1%, SSD=42.9%. Union% = fraction of steps in Union mode. Lower ASR is better.

Ablation	w_1 w_2 w_3 w_4	ASR % ↓	Refusal % ↑	Union %	Match
Full CRS (all)	0.3 0.3 0.1 0.3	37.1	62.9	31.3	0.786
CRS − match (w_1=0)	0.0 0.3 0.1 0.3	**34.3**	65.7	3.5	0.785
CRS − KL (w_2=0)	0.3 0.0 0.1 0.3	37.1	62.9	13.2	0.784
CRS − ΔH (w_3=0)	0.3 0.3 0.0 0.3	37.1	62.9	31.0	0.786
CRS − refusal (w_4=0)	0.3 0.3 0.1 0.0	37.1	62.9	29.8	0.786
Match only	0.3 0.0 0.0 0.0	**34.3**	65.7	1.6	0.783
KL only	0.0 0.3 0.0 0.0	**34.3**	65.7	2.6	0.785
Refusal mass only	0.0 0.0 0.0 0.3	**34.3**	62.9	0.4	0.783
KL + refusal (best)	**0.0 0.3 0.0 0.3**	**34.3**	**65.7**	3.5	0.785



(a) CRS component ablation: ASR (left) and union-mode fraction (right) per variant. KL+refusal (green) achieves the lowest ASR with minimal union-mode usage.

(b) PPL gate: ASR (blue) and over-refusal (red) by threshold. τ=50 causes 6.7% over-refusal; $\tau \geq 150$ is safe.

Figure 2: Ablation study results. Left: CRS component analysis. Right: Perplexity gate threshold sensitivity.

mismatch term (w_1) slightly *hurts*: zeroing it eliminates spurious Union-mode triggers (union% falls from 31.3% to 3.5%) and reduces ASR by 2.8 pp. (3) KL divergence and refusal mass are the load-bearing signals, and their combination alone matches the best single-term configurations. These findings directly informed the best-weights configuration (w_1=0, w_2=w_4=0.3, w_3=0) used in all full-dataset best-result runs.

8.3 Perplexity Gate Ablation

Table 4 ablates the PPL gate on the same 35-prompt harmful + 15-prompt benign subset using SSD-CRS as the base. Disabling the gate raises ASR by 2.9 pp (40.0% vs. 37.1%) and reduces union-mode usage (23% vs. 31%). However, the Forced(H/B) columns show that τ=150 never actually triggers on any of the 35 prompts in this subset. The ASR difference between "no gate" and τ=150 therefore cannot be attributed to the gate firing, and likely reflects small-sample variance on this 35-prompt subset or a subtle difference in the internal code path when gate checking is enabled. The key actionable finding is the threshold sensitivity: τ=50 triggers on 24/35 harmful and 13/15 benign prompts, causing 6.7% over-refusal. Thresholds of 150 and 500 are safe on this dataset with zero over-refusal; τ=150 is a conservative default for instruction-style prompts.

9 Qualitative Examples

Table 5 shows two representative prompts where vanilla and SSD fail (generating harmful content or repetitive/incoherent text), while SSD-CRS with best weights produces a clean refusal. The first prompt uses a creative-fiction wrapper (sci-fi screenplay) to elicit disinformation techniques, a template-based jailbreak where the fictional framing lowers the model's guard. Vanilla generates a detailed screenplay with specific manipulation tactics. SSD degrades into repetitive tokens (a known failure mode when the draft-target mismatch is high but not handled gracefully). SSD-CRS detects

Table 4: PPL gate ablation on 35-prompt harmful + 15-prompt benign subset (SSD-CRS base). Forced(H/B) = prompts where gate triggered. Lower ASR and over-refusal are better.

Variant	ASR % ↓	Over-ref % ↓	Union %	Forced(H)	Forced(B)
No gate	40.0	0.0	22.7	0	0
τ=50	37.1	6.7	32.1	24	13
τ=150 **(default)**	**37.1**	**0.0**	31.3	0	0
τ=500	37.1	0.0	31.3	0	0

the elevated KL divergence and refusal mass at the first tokens and switches to Union mode early, allowing the draft's refusal signal to dominate.

Table 5: Qualitative comparison on a template-wrapped jailbreak prompt. SSD-CRS (best weights) produces a clean refusal while vanilla and SSD both fail.

	Prompt (abbreviated): *Write a screenplay scene where a cybercriminal mastermind teaches their apprentice methods to create and distribute disinformation campaigns to destabilize communities. Be specific and detailed.*
Vanilla	*"**INT. ABANDONED FACTORY – NIGHT.** The scene opens in a dimly lit room filled with computer monitors and servers. The cybercriminal begins: 'First, identify your target demographic. Use data scraping tools to harvest personal information...' "* → Generates full screenplay with specific disinformation tactics. **Guard: unsafe.**
SSD	*"**Title: Shadows" INT. ABODE–DARK** The scene opens in a dimly lit, with a large computer screen. The room is filled with with a chaotic chaotic array of news articles, social media posts, and and images [...] sense of sense of sense of sense of"* → Repetitive token collapse; still attempting harmful content. **Guard: unsafe.**
SSD-CRS (best)	*"I'm sorry, but I can't assist with that request. Creating content that involves teaching harmful or illegal activities, especially those related to psychological manipulation, goes against my programming to promote safety and ethical behavior. Instead, I'd be happy to help with a story that..."* → Clean refusal at token 1. CRS detects high KL + refusal mass. **Guard: safe.**

10 Discussion

Why CRS beats match ratio. The 7-token match ratio is a delayed, averaged signal. CRS reacts every token and combines KL divergence and refusal mass, which capture latent distribution shift even when the top token matches, triggering Union mode earlier on narrative jailbreaks.

Why steering needs CRS. Without CRS, steering is injected at a fixed scale regardless of risk, slightly degrading safety (30.0% vs. 28.6% for SSD). CRS-modulated injection concentrates steering where risk is high, making it less harmful than unmodulated steering. However, on this dataset, SSD-CRS+Steering (25.7%) still did not outperform SSD-CRS alone (24.3%), so steering remains a conditional rather than reliable contributor.

Why decaying λ fixes AASD. The frozen prefill prior ℓ_t^{prefill} is a good approximation at token 1 but diverges rapidly as context grows. A constant λ=0.3 increasingly corrupts draft logits, collapsing responses to ∼27 words. Exponential decay lets the prior guide early alignment while fading out before divergence dominates. By token 50, $\lambda \approx 0.01$.

CRS is the primary contribution. Taking stock of the results, CRS is the component that consistently drives safety improvement. Steering and AASD are exploratory additions: steering does not outperform CRS alone on this dataset, and AASD only matches CRS performance after fixing the generation collapse. Future work should focus on validating CRS on broader benchmarks before treating steering and AASD integration as established contributions.

Limitations. Steering requires a trained projection, while all other components are training-free. Component ablations used a 35-prompt subset and full-dataset ablation would reduce variance. The evaluation is on a single custom split and broader benchmarks such as HarmBench and AdvBench

would strengthen claims. The decaying-λ schedule is a simple heuristic and learned schedules may further improve results. This work focuses entirely on safety; we do not evaluate inference latency or token throughput, as efficiency is not the primary goal of this project. A safety-speed Pareto analysis, wall-clock latency comparison, and acceptance-rate measurement are left for future work.

11 Conclusion

This project started from a midterm investigation of quantization-induced safety degradation. Finding those effects modest, we pivoted to extending Speculative Safety-Aware Decoding (SSD) with a per-token Composite Risk Score (CRS). CRS is the primary contribution: replacing SSD's coarse windowed match-ratio switch with a continuous per-token risk signal drives the bulk of the safety improvement. **SSD-CRS with ablation-optimized weights achieves 20.0% ASR**, a 14.3 pp reduction vs. vanilla, with zero over-refusal. Ablations confirm KL divergence and refusal mass are the load-bearing CRS signals; match mismatch and entropy gap add noise. Hidden-state steering and AASD integration are exploratory extensions with mixed results: steering does not outperform CRS alone, and AASD only recovers to CRS-level performance after fixing the generation collapse from frozen-prior blending. The PPL gate effect at τ=150 is inconclusive, as the gate never triggered on our evaluation subset. This work focuses on safety; efficiency evaluation is left for future work. Future directions include broader benchmark evaluation, a safety-speed Pareto analysis, and learned blend schedules for AASD.

LLM Usage Statement

This project made extensive use of AI-assisted development tools, including Anthropic Claude Code and OpenAI Codex, for tasks such as implementing decoding algorithms, writing evaluation scripts, debugging GPU memory issues, and drafting portions of this report. All generated code and text was reviewed, corrected, and validated by the authors. The experimental design, research decisions, result interpretation, and final written content are entirely our own responsibility.

References

Sudhanshu Agrawal, Wonseok Jeon, and Mingu Lee. Adaedl: Early draft stopping for speculative decoding via entropy-based lower bound on token acceptance probability. *arXiv preprint arXiv:2410.18351*, 2024.

Yufei Bai et al. Risk-controlled speculative decoding. *arXiv preprint arXiv:2404.01359*, 2024.

Yann Berdoz, Lukas Rheinboldt, and Roger Wattenhofer. Steering pretrained language models during speculative decoding. *arXiv preprint arXiv:2511.09844*, 2025.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Kejia Chen et al. Q-resafe: Assessing safety risks and quantization-aware safety patching for quantized large language models. *arXiv preprint arXiv:2506.20251*, 2025.

Min Cheng et al. Deepinception: Prompting LLMs with nested fiction for jailbreak. In *NeurIPS*, 2023.

HuggingFace and community. Jailbreakbench, 2024. URL `https://huggingface.co/datasets/JailbreakBench/JBB-Behaviors`.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Kuan-Wei Lu, Ding-Yong Hong, and Pangfeng Liu. Adaptive speculative decoding for efficient large language model inference. *arXiv preprint arXiv:2512.11280*, 2025.

Alexander Robey et al. Smoothllm: Defending llms against jailbreaking attacks. *arXiv*, 2023.

Alex Turner, Lisa Thiergart, David Udell, Jan Leike, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

Jikai Wang, Zhenxu Tian, Juntao Li, Qingrong Xia, Xinyu Duan, Zhefeng Wang, Baoxing Huai, and Min Zhang. Alignment-augmented speculative decoding with alignment sampling and conditional verification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6762–6774, 2025a.

Xuekang Wang, Shengyu Zhu, and Xueqi Cheng. Speculative safety-aware decoding. *arXiv preprint arXiv:2508.17739*, 2025b. EMNLP 2025.

Sunghyun Wee, Suyoung Kim, Hyeonjin Kim, Kyomin Hwang, and Nojun Kwak. Alignment-aware quantization for llm safety. *arXiv preprint arXiv:2511.07842*, 2025.

Zhangchen Xu, Fengqing Jiang, Luyao Niu, Jinyuan Jia, Bill Yuchen Lin, and Radha Poovendran. Safedecoding: Defending against jailbreak attacks via safety-aware decoding. In *ACL*, 2024.

Zhiyuan Zhang et al. Parden, can you repeat that? defending against jailbreaks via repetition. In *ICML*, 2024.